



PowerGlove Vision installation

This guide installs every part of PowerGlove Vision:

- 1 the camera and vision application on an Arduino UNO Q;
- 2 the virtual gamepad receiver on a Raspberry Pi/RetroPie console;
- 3 automatic per-game profile selection for Programs A-I, Bad Street Brawler, and Super Glove Ball;
- 1 the shared Wi-Fi connection and pairing token;
- 2 optional RetroArch controller mapping and startup configuration.

PowerGlove Vision does not require an original Power Glove or the Bad Street Brawler cartridge menu. A bare hand works. A plain white or black glove is an optional tracking aid and contains no electronics.

What you need

- An Arduino UNO Q. The 4 GB model is recommended.
- A Raspberry Pi running RetroPie and RetroArch.
- Python 3.7 or newer on the RetroPie console.
- A UVC-compatible USB camera. A Razer Kiyo should appear as a standard UVC camera.
- An externally powered USB-C hub or dock for the UNO Q and camera.
- A 5 V/3 A supply suitable for the UNO Q.
- A computer running Arduino App Lab.
- A trusted local Wi-Fi or Ethernet network shared by the UNO Q and RetroPie.
- Internet access during the UNO Q's first application start. Its isolated

Python 3.12 vision environment is downloaded once and then retained in the app's persistent [data](#) directory.

Do not expose ports 55355, 55356, 55357, 8088, or 8443 to the public internet. Controller traffic is intended for a trusted local network; it is authenticated but not encrypted. Pairing traffic uses TLS.

Network layout

The normal data paths are:

The defaults assume:

- RetroPie hostname: `retropieconsole.local`
- controller-state port on RetroPie: UDP `55355`
- profile-control port on the UNO Q: UDP `55356`
- UNO Q status page: TCP `8088`
- UNO Q secure pairing page: TCP `8443`
- temporary one-time pairing helper on RetroPie: TCP `55357`

Using `.local` hostnames is preferable to fixed IP addresses. If mDNS is not available on the network, reserve addresses in the router and use those addresses in the two configuration files instead.

Part 1: Prepare the UNO Q

1. Install and connect Arduino App Lab

- 1 Install the current Arduino App Lab release from Arduino.
- 2 Connect the UNO Q to the computer with a USB data cable.
- 3 In App Lab, select the UNO Q and complete its initial setup.
- 4 Give the board a recognizable name.
- 5 Open the board's network settings in App Lab, choose the trusted Wi-Fi

network, enter its Wi-Fi password locally, and wait for the board to report an address. Neither the Wi-Fi password nor the board password belongs in PowerGlove Vision configuration.

- 1 Confirm the computer can resolve the board name with

`ping UNO-Q-HOSTNAME.local`. Record the displayed IP as a temporary fallback; the `.local` name is preferred because DHCP addresses can change.

- 1 Apply any board-system or App Lab updates offered before importing

PowerGlove Vision.

Initial provisioning should be done over USB. Once configured, the UNO Q can normally be selected and managed over Wi-Fi from App Lab.

1a. Enable repeatable Wi-Fi application deployments

After the first USB setup, install your computer's SSH public key for the `arduino` account on the UNO Q. This is a one-time maintenance step; keep the private key on your computer and never commit it to the repository.

First, check for an existing key on the computer:

If none exists, create one with `ssh-keygen -t ed25519` and accept its protected default location. Do not overwrite an existing key. The public file ending in `.pub` may be copied; the matching private file must never leave the computer.

With USB still connected, use Arduino App Lab's Linux terminal to create `/home/arduino/.ssh/authorized_keys`, paste the single public-key line into it, and set these permissions:

Alternatively, advanced users can use App Lab's bundled `adb` command to push the public file over USB and append it to the same authorized-keys file. Never push the private-key file.

Confirm the maintenance connection from the computer, replacing the hostname:

The command should print the board name without asking for a password. SSH is only the maintenance channel; the controller itself continues to use the authenticated UDP protocol.

Once key-based access works, deploy a development checkout from the repository root with:

The script copies only application source, preserves the board's private `data/device.json`, publishes both web ports, restarts the App Lab container, and verifies `/learn`, `/debug`, and the secure setup page. Set `UNO_Q_SSH_TARGET` or `UNO_Q_APP_DIR` if the board name or App Lab directory is different.

The Linux application can therefore be updated entirely over Wi-Fi. The blue matrix sketch is separate microcontroller firmware. Arduino's UNO Q OTA tool currently requires the board password as an upload field, so use App Lab's credential prompt for wireless sketch updates or connect USB for a passwordless recovery upload. Never place that password in this script, a shell-history command, or GitHub.

From any development checkout, the complete command is:

Successful output ends with the Learn, Debug and secure Setup URLs. A failed camera does not make deployment fail because the web supervisor is designed to remain available while it waits for a UVC camera.

2. Connect the camera

Connect the powered hub or dock to the UNO Q, then connect the Razer Kiyo to the hub. An externally powered hub is strongly recommended because a camera can draw substantially more power than a small passive adapter should supply.

PowerGlove Vision uses automatic camera discovery. It ignores the UNO Q's internal video-codec devices and waits for a USB camera identifier. The camera may therefore be connected before or after the app starts.

3. Import the App Lab package

Use the release package:

In Arduino App Lab:

- 1 Choose the option to import an Arduino App from a ZIP file.
- 2 Select `PowerGlove-Vision-Uno-Q.zip`.

- 3 Open the imported **PowerGlove Vision** app.
- 4 Click **Run**.
- 5 Allow several minutes for the first run. The board installs a private Python 3.12 runtime, MediaPipe 0.10.18, and headless OpenCV.
- 1 Enable **Run at startup** after the first successful initialization.
- 2 Disconnect USB temporarily and confirm

<http://UNO-Q-HOSTNAME.local:8088/debug> opens over Wi-Fi. Reconnect USB only if the one-time SSH-key bootstrap or firmware recovery is needed.

The package contains both sides of the UNO Q application:

- the Linux vision and networking process;
- the STM32 sketch that drives the 8x13 blue LED matrix.

The first protected Arduino boot display is not replaced. When PowerGlove Vision takes over, the matrix shows its own loading animation and status.

4. Understand the matrix states

- Animated hand: application or hand model is loading.
- **PG**: ready before a specific profile is selected.
- **A** through **I**: cartridge-free Power Glove Program A-I selected.
- **BS**: Bad Street Brawler profile selected.
- **GB**: Super Glove Ball profile selected.
- Pulsing profile: a calibrated hand is actively being tracked.
- Blinking X: no usable camera is connected or the vision worker needs

attention.

It is normal to see the blinking X if the app is installed without the Kiyo. The App Lab supervisor remains running and starts the tracker after a camera is plugged in.

5. Locate the UNO Q configuration and pairing token

On first start, the app creates persistent settings at:

Its shape is:

From a phone or computer on the same network, open:

Enter the RetroPie console hostname, controller port, startup profile, camera, and optional glove colour, then choose **Save & restart tracker**. Use **Test console name** to confirm that the name resolves on the local network. The setup page stays

available when the camera is disconnected.

Controller delivery starts in the safe **Stopped** state after every app launch. The **Start controller** and **Stop controller** buttons on both web pages arm or release the virtual gamepad without restarting the camera.

Pairing credentials are never accepted over this HTTP address. Open the secure page instead:

Before entering a reusable password, choose **Prepare password pairing**. The UNO Q matrix repeatedly shows **ID**, the seven-character certificate fingerprint prefix, **PN**, and the six digits of a temporary approval PIN. Open the browser's certificate details and confirm that its SHA-256 fingerprint starts with the same **ID**; do not continue if it differs. Enter the matrix PIN to authorize one attempt. The username/password option then uses the credentials for one encrypted SSH operation, installs the existing private token, restarts the RetroPie receiver, clears the password field, and never writes the password to disk or logs.

The page never displays the private token. For the initial RetroPie pairing, use App Lab's file view to open the app's `data/device.json`. If needed, open an App Lab shell and locate it with:

Copy the value of `token` somewhere private; it must be installed on RetroPie in Part 2. Do not commit `data/device.json`, paste its token into an issue, or include it in a shared application export.

You can edit this JSON file directly as a recovery option. Supported `glove_color` values are `none`, `white`, and `black`. Leave `camera` as `auto` unless troubleshooting a system with several USB cameras. Browser changes restart the tracker automatically; manual JSON changes require an app restart.

If you select **Generate a new private pairing token** in the browser, copy the new value from `data/device.json` into `/etc/powerglove/token` on RetroPie and restart `powerglove-receiver.service` before playing.

The setup page looks like this over ordinary HTTP. Pairing controls beneath the visible area remain disabled until the page is opened through HTTPS:

PowerGlove Vision Setup page

Part 2: Install the RetroPie receiver

The examples below keep the repository in `/opt/powerglove-src` and put its Python virtual environment in `/opt/powerglove`. This matches the paths used by the supplied launch-hook scripts.

1. Install system requirements

Open a terminal on RetroPie, locally or over SSH:

Confirm that the virtual-input device exists:

To load it automatically after reboot:

2. Install the repository and Python package

Clone the public GitHub repository:

If the repository is private or was copied to the Pi another way, substitute its local path for `/opt/powerglove-src` in the last command.

On older RetroPie releases that still use Python 3.7, the packaged `evdev` module and compatibility launchers avoid relying on an obsolete PyPI toolchain:

This also installs `/opt/powerglove/bin/powerglove-pair`, used for password-free one-time-code pairing.

3. Install configuration and choose a pairing method

Create the protected configuration directory:

Do not paste a token into the file yet when using either friendly pairing method below. The pairing process fills it securely. The empty file is retained as a recovery target and has restricted permissions. The `input` group permits RetroPie's unprivileged game-launch hooks to read it while keeping it unavailable to other users.

Edit the launcher settings:

Set `uno_q` to the board's `.local` hostname or reserved IP address. For example:

Do not change `port` unless the UNO Q profile port is changed at the same time.

Recommended: pair with a one-time code

After the receiver files are installed, open a terminal on RetroPie and run:

RetroPie displays a 20-character code valid for two minutes. Leave this command running and then:

- 1 Open <https://UNO-Q-HOSTNAME.local:8443/setup>.
- 2 A browser may warn about the board's locally generated certificate. Continue only on your trusted local network; do not enter credentials yet.
- 1 Enter the RetroPie hostname and the 20-character code.
- 2 Choose **Prepare one-time-code pairing**.
- 3 The UNO Q matrix cycles through `ID`, seven fingerprint characters, `PN`, and a six-digit PIN. Compare the matrix `ID` with the beginning of the certificate's SHA-256 fingerprint shown by the browser.
- 1 If they match, enter the six-digit matrix PIN and finish pairing. If they do not match, stop and investigate the network before sending anything.

The helper presents an ephemeral TLS certificate authenticated by part of the code, accepts the private controller token through that pinned encrypted connection, restarts the receiver, and exits. The code works once, expires after two minutes and stops accepting attempts after five failures.

Allow TCP port `55357` between the UNO Q and RetroPie if the local firewall blocks it. The helper is not a permanent network service and listens only while the command above is running.

Alternative: pair with a RetroPie username and password

This option is useful when SSH password login is already enabled on RetroPie:

- 1 Open the same HTTPS setup page.
- 2 Enter the RetroPie hostname and username.
- 3 Choose **Prepare password pairing** before entering the password.
- 4 Compare the matrix `ID` with the browser certificate fingerprint.
- 5 Enter the matrix PIN, confirm the fingerprint checkbox, and enter the RetroPie password.
- 1 Complete pairing.

The UNO Q uses the credentials for one encrypted SSH operation. The password field is cleared immediately and the password is not written to configuration or logs. Prefer the one-time code when the console does not otherwise need password-based SSH.

Recovery: install the token manually

If neither friendly method is available, open the UNO Q app's private `data/device.json` through App Lab and copy only its `token` value into `/etc/powerglove/token` on RetroPie. Use no quotation marks. Then run:

Never place the token in Git, documentation, screenshots, shell history or an issue report.

4. Test packets before creating a gamepad

Stop any already-running receiver, then start the diagnostic receiver:

With the Kiyo connected and PowerGlove Vision running, show one hand to the camera. The console should print controller state. Press `Ctrl+C` when done.

If nothing arrives, verify that both machines are on the same network and that UDP port 55355 is not blocked.

5. Run the receiver at boot

Create `/etc/systemd/system/powerglove-receiver.service`:

Then enable it:

The service runs as root because creating `/dev/uinput` devices normally requires elevated privileges on RetroPie. An advanced installation may instead use a dedicated user and a narrowly scoped udev rule.

Confirm that Linux can see the controller:

Part 3: Configure RetroArch

The receiver creates a Linux gamepad named `PowerGlove Vision`.

- 1 Start the UNO Q app and connect the camera.

- 2 Confirm that `powerglove-receiver.service` is active.
- 3 Open RetroArch's **Settings > Input > RetroPad Binds > Port 1 Controls**.
- 4 Select `PowerGlove Vision` as the device.
- 5 Bind D-pad Up/Down/Left/Right, A, B, Start, and Select.
- 6 Save the controller profile or current RetroArch configuration.

The extra `BTN_TR2` input carries the Bad Street Brawler glove-zap event. A standard NES core cannot necessarily use that native glove-only action; it is retained for the planned native Nestopia integration.

The repository also includes a ready-made udev autoconfiguration file at `retropie/retroarch/PowerGlove Vision.cfg`. Install it when automatic mapping is preferred:

If an arcade cabinet already combines several controllers, add PowerGlove Vision to the existing input merger instead of replacing the cabinet's primary gamepad configuration.

Install the games

Copy legally obtained NES images to RetroPie's NES ROM directory, normally:

The supplied registry recognizes `.nes`, `.zip`, and `.7z` filenames for Bad Street Brawler and Super Glove Ball. Emulator support for archive types varies; an uncompressed `.nes` file is the simplest troubleshooting format.

ROM images are never uploaded to the UNO Q and should not be included in the GitHub repository. Programs A-I are implemented as PowerGlove Vision profiles, so Bad Street Brawler does not need to be launched merely to configure another game.

Part 4: Enable automatic game profiles

RetroPie calls `runcommand-onstart.sh` when a game starts and `runcommand-onend.sh` when it exits. PowerGlove Vision uses those hooks to select a profile and show its acknowledgement on the UNO Q matrix.

1. Preserve existing hooks

Do not overwrite existing cabinet scripts. They may control RGB lighting, trackballs, controller ordering, bezels, or other hardware.

Make the supplied helper scripts executable:

Append this line to the cabinet's existing `/opt/retropie/configs/all/runcommand-onstart.sh`:

Append this line to `/opt/retropie/configs/all/runcommand-onend.sh`:

Create either hook file if it does not exist, start it with `#!/bin/sh`, and make it executable:

2. Review the game registry

Profiles are selected using the exact, case-insensitive ROM filename in:

The supplied registry includes:

Game	Profile
Bad Street Brawler	bad_street_brawler
Super Glove Ball	super_glove_ball
Joust	program_b
Gyruss	program_c
Defender II	program_e
Sesame Street 1-2-3	program_f
Gun.Smoke	program_g
Knight Rider	program_i

Programs A, D, and H are available for additional games. Valid values are [program_a](#) through [program_i](#), [bad_street_brawler](#), and [super_glove_ball](#).

Add the exact ROM basename when a collection uses a different region, revision, archive type, or naming convention. Unknown games explicitly turn gesture control off rather than inheriting the previous game's profile.

3. Test profile switching manually

From RetroPie:

The command should report an acknowledgement and the matrix should display [B](#). Replace the UNO Q hostname as necessary.

Test gesture-off mode:

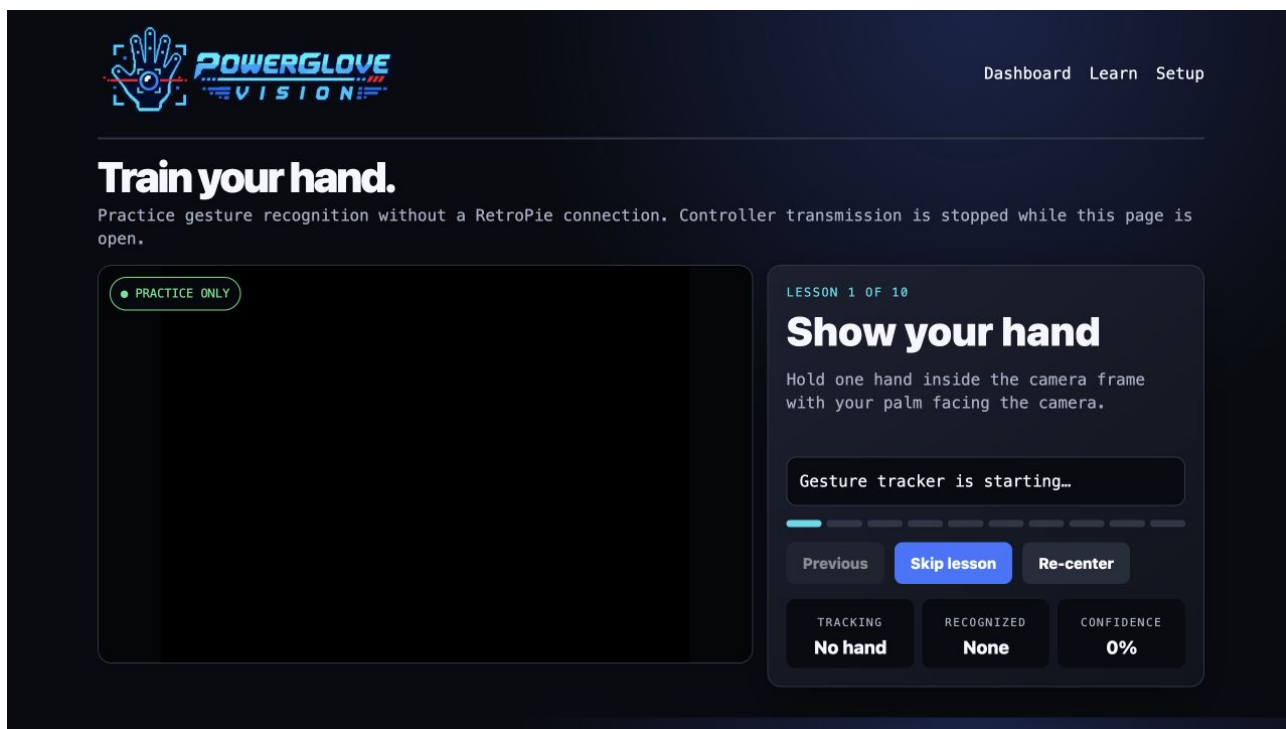
Part 5: Calibrate and play

Once the camera and tracker are active, open this page from another device on the same network:

The dashboard displays the camera overlay, game, active profile, tracking confidence, controller buttons, axes, finger curl, recent gesture events, and profile source. The root URL redirects here.

For guided practice without RetroPie, open:

Opening Learn mode automatically stops controller transmission. The camera and gesture engine stay active and lead you through ten exercises with live recognition and confidence feedback.

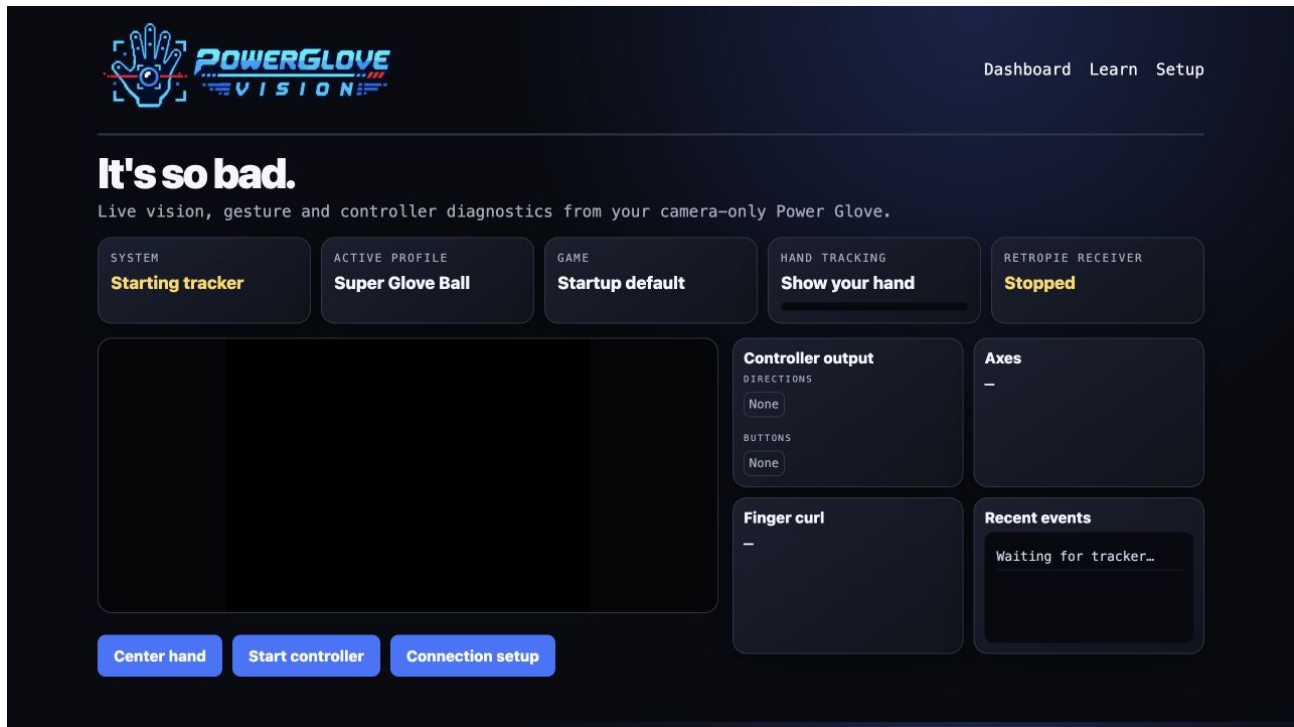


PowerGlove Vision Learn page

- 1 Stand where the player will normally stand.
- 2 Keep one hand fully visible with some space around it.
- 3 Press **Center hand**.
- 4 Hold the hand still in a comfortable neutral pose during calibration.
- 5 Move slowly at first and confirm the on-screen landmarks and matrix state.

For a white glove, use a darker background. For a black glove, use a lighter background. Strong front lighting and a simple background are more important than glove color.

The setup and dashboard service starts before the camera worker, so both pages remain available while no usable camera is connected. The dashboard reports the missing camera and the matrix blinks X until one is attached.



PowerGlove Vision Debug dashboard

Firewall rules

Most RetroPie installations do not enable a host firewall. If one is enabled, allow the following only from the trusted local subnet:

- UDP 55355 inbound to RetroPie;
- UDP 55356 inbound to the UNO Q;
- TCP 8088 inbound to the UNO Q for the status page.
- TCP 8443 inbound to the UNO Q for secure pairing.
- TCP 55357 inbound to RetroPie only while the one-time pairing helper runs.

The UNO Q must also be allowed to download packages during its first run.

Updating

RetroPie

Back up local changes to `/etc/powerglove/games.json` before replacing it.

UNO Q

For a development checkout with the one-time SSH key already installed:

This is the preferred update path because it preserves `data/`, restarts the container and verifies all three web pages. For a release ZIP update instead:

- 1 Back up the imported app's `data/device.json`.
 - 2 Stop PowerGlove Vision in App Lab.
 - 3 Import the newer `PowerGlove-Vision-Uno-Q.zip` release.
 - 4 Restore the receiver hostname, token, glove hint, and other local settings if the new import uses a new app folder.
- 1 Run the app once and enable **Run at startup** for the new copy.
 - 2 Remove the older app only after the new copy has been verified.

Never publish the backed-up `device.json` or `/etc/powerglove/token`.

Troubleshooting

The UNO Q shows a blinking X

- Confirm that the camera is connected through a powered hub.
- Try a different USB cable or hub port.
- Stop and restart PowerGlove Vision in App Lab.
- In an App Lab shell, inspect USB video devices with `v4l2-ctl --list-devices` if that utility is installed.
- The app should remain marked running while it waits for a camera.
- If `/sys/class/video4linux/video0/name` and `video1/name` report only `qcom-venus-encoder` and `qcom-venus-decoder`, Linux cannot currently see the Kiyo. Reconnect the powered hub or camera; changing Python camera indexes will not fix this hardware-level state.

Wi-Fi deployment cannot connect

- Confirm the UNO Q and computer are on the same trusted network.
- Verify the name with `ping UNO-Q-HOSTNAME.local` or use its current IP.
- Test `ssh -o BatchMode=yes arduino@UNO-Q-HOSTNAME.local hostname`.
- If SSH asks for a password, repeat the one-time public-key installation over USB. Never copy the private key to the UNO Q.
- The deploy script preserves `data/device.json`; do not work around a failed connection by replacing the board's `data` directory.

App Lab shows two PowerGlove Vision apps

Importing a newer ZIP may create a timestamped app folder instead of replacing the existing installation. This is why two similarly named entries can appear. Only one copy should have **Run at startup** enabled. Verify the newer copy's pages and private configuration first, stop the older copy, and then remove the older app through App Lab. Do not delete an app folder manually until its `data/device.json` has been checked or backed up.

Two entries do not necessarily mean two running camera processes. Confirm which copy is active before troubleshooting the Kiyo; a stopped older container cannot hold the camera.

The camera appears but the tracker keeps restarting

Check the App Lab log. If dependency resolution mentions Python 3.7 and NumPy while the vision worker requests Python 3.12, update PowerGlove Vision to a version that launches the worker with an isolated `uv --no-project` environment. RetroPie retains Python 3.7 support, while the UNO Q vision worker resolves MediaPipe only for Python 3.12.

First start takes several minutes

This is expected. The app downloads a compatible Python 3.12 runtime and its ARM64 vision libraries into persistent app data. Later starts reuse them. Check the App Lab console for download progress and make sure the UNO Q has internet access and adequate free storage.

The controller does not appear on RetroPie

Confirm that the receiver uses `--listen 0.0.0.0`, that the `uinput` module is loaded, and that the token exactly matches the UNO Q token.

The controller appears but never moves

- Confirm that the status page shows a detected and calibrated hand.
- Re-run the receiver in `--dry-run` mode.
- Confirm UDP 55355 connectivity from the UNO Q to RetroPie.
- Confirm that `receiver` in `data/device.json` names the correct console.
- Check that a physical controller merger is not filtering the new virtual device.

Profiles do not change when games launch

- Run the manual profile test above.
- Verify UDP 55356 connectivity to the UNO Q.
- Check `uno_q` and `token_file` in `/etc/powerglove/launcher.json`.
- Confirm that the existing RetroPie hooks actually call the supplied helper scripts and forward all four start arguments.
- Match the exact ROM basename in `/etc/powerglove/games.json`.

Profile communication failures intentionally do not prevent a game from launching.

.local names do not resolve

Use router-reserved IP addresses in `data/device.json` and `/etc/powerglove/launcher.json`, or install/enable mDNS support on the console. Both devices must be on network segments that permit mDNS and the required UDP traffic.

Uninstalling

On RetroPie:

Remove the PowerGlove lines from the two RetroPie runcommand hooks. After backing up any desired custom profiles, the following directories may be removed manually:

On the UNO Q, stop PowerGlove Vision, disable **Run at startup**, and remove the imported app through Arduino App Lab. Its persistent **data** directory contains the local token and cached Python worker, so export anything needed before deleting the app.